

Intelligent Disease Prediction System Using Machine Learning

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: df = pd.read_csv(r'D:/ML Project/diabetes.csv')
df
```

```
Out[2]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33
...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63
764	2	122	70	27	0	36.8	0.340	21
765	5	121	72	23	112	26.2	0.245	30
766	1	126	60	0	0	30.1	0.349	47
767	1	93	70	31	0	30.4	0.315	28

768 rows × 9 columns

Understand the data

```
In [3]: df.head()
```

```
Out[3]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	32
3	1	89	66	23	94	28.1	0.167	21
4	0	137	40	35	168	43.1	2.288	33

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 768 entries, 0 to 767
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

```
dtypes: float64(2), int64(7)
```

```
memory usage: 54.1 KB
```

```
In [5]: df.describe()
```

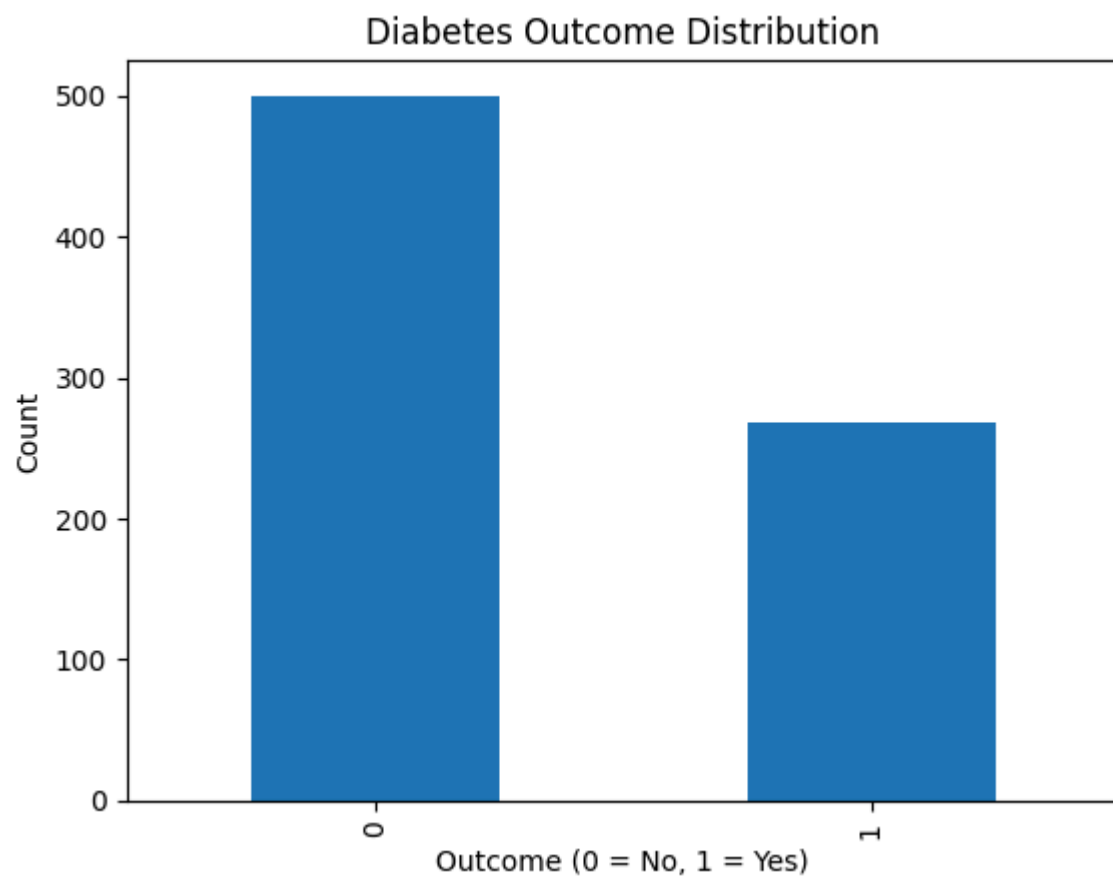
```
Out[5]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471471
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.336133
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.337500
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.497500
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.622500
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	0.997500



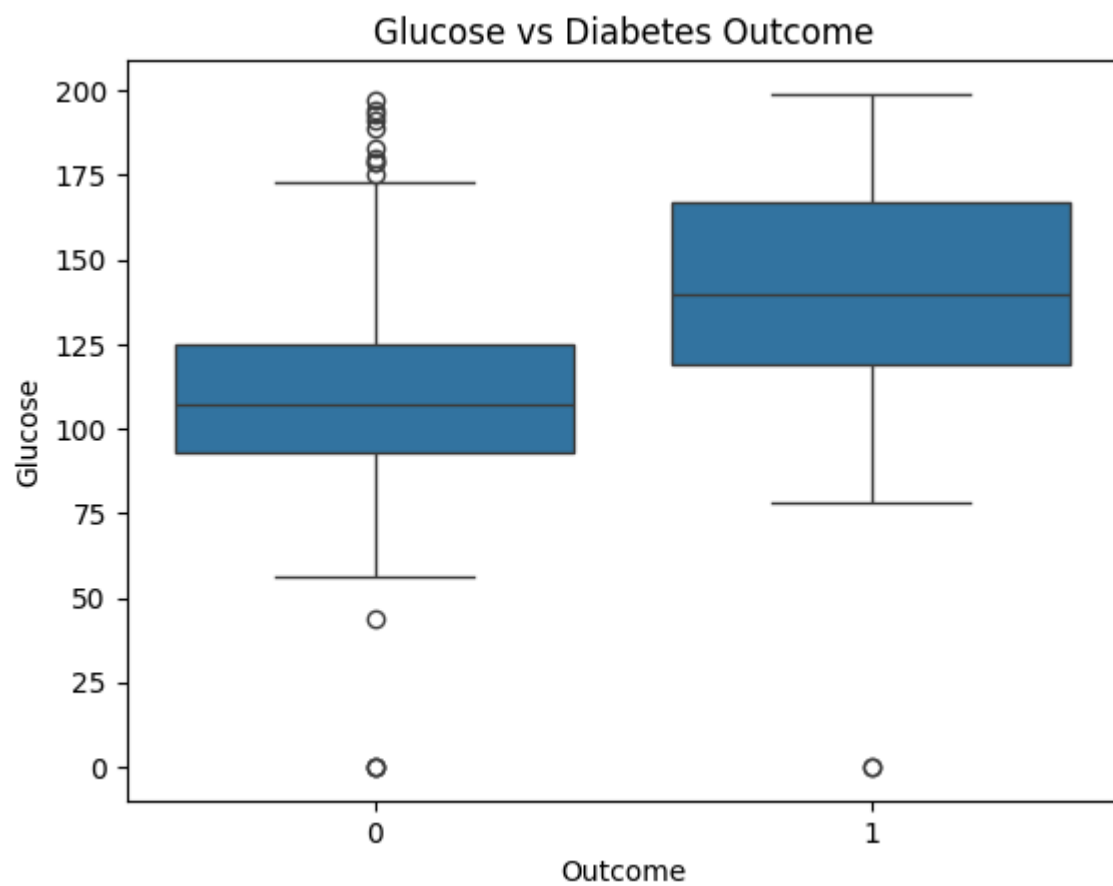
```
In [6]: import matplotlib.pyplot as plt

df['Outcome'].value_counts().plot(kind='bar')
plt.xlabel('Outcome (0 = No, 1 = Yes)')
plt.ylabel('Count')
plt.title('Diabetes Outcome Distribution')
plt.show()
```

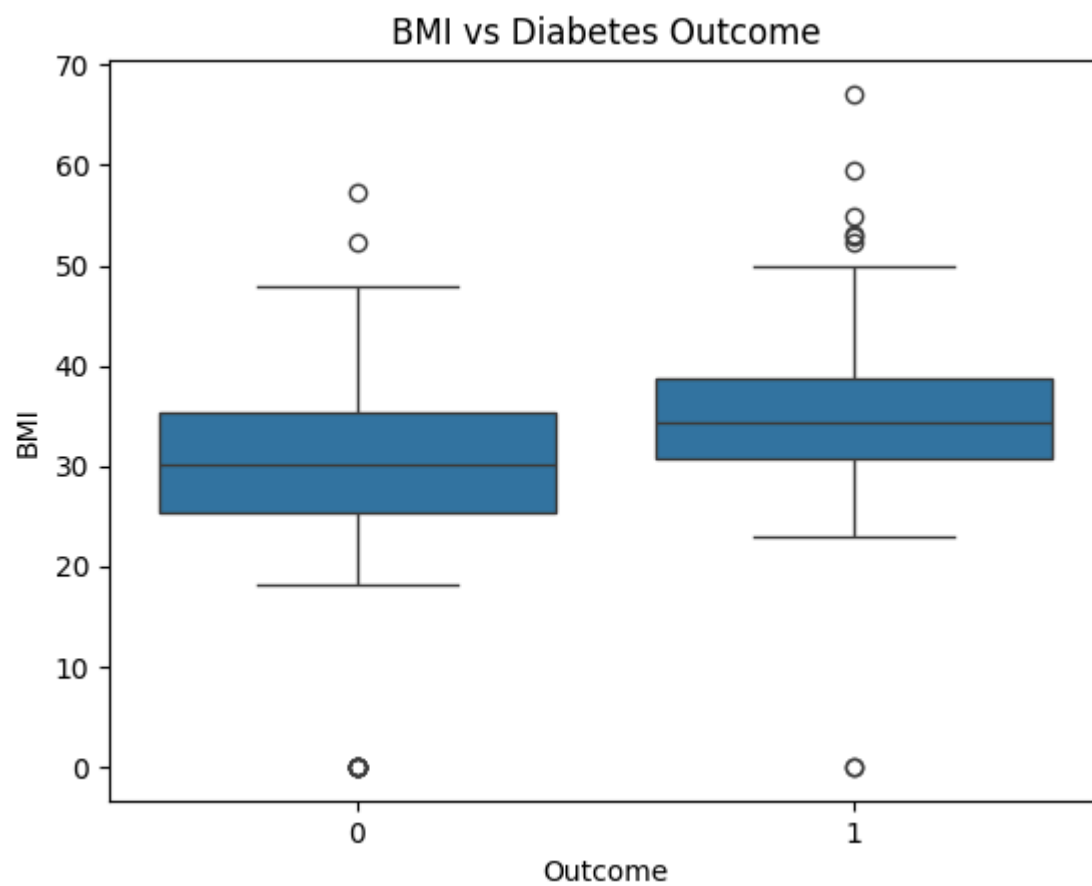


```
In [7]: import seaborn as sns

sns.boxplot(x='Outcome', y='Glucose', data=df)
plt.title('Glucose vs Diabetes Outcome')
plt.show()
```



```
In [8]: sns.boxplot(x='Outcome', y='BMI', data=df)
plt.title('BMI vs Diabetes Outcome')
plt.show()
```



```
In [9]: df.shape
```

```
Out[9]: (768, 9)
```

Cleaning the Data

```
In [10]: df.isnull().sum() #all are 0 so its a good database / it doesnt contain null value
```

```
Out[10]: Pregnancies      0
Glucose      0
BloodPressure  0
SkinThickness  0
Insulin      0
BMI          0
DiabetesPedigreeFunction  0
Age          0
Outcome      0
dtype: int64
```

```
In [11]: df
```

Out[11]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	0	33.6	0.627	50
1	1	85	66	29	0	26.6	0.351	31
2	8	183	64	0	0	23.3	0.672	33
3	1	89	66	23	94	28.1	0.167	24
4	0	137	40	35	168	43.1	2.288	33
...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63
764	2	122	70	27	0	36.8	0.340	25
765	5	121	72	23	112	26.2	0.245	30
766	1	126	60	0	0	30.1	0.349	41
767	1	93	70	31	0	30.4	0.315	23

768 rows × 9 columns

In [12]:

df['Insulin'] = df['Insulin'].replace(0,np.nan) *#replace 0 to NAN*

In [13]:

df['Insulin'].isnull().mean() * 100

Out[13]:

np.float64(48.69791666666667)

In [14]:

df

Out[14]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	NaN	33.6	0.627	50
1	1	85	66	29	NaN	26.6	0.351	31
2	8	183	64	0	NaN	23.3	0.672	33
3	1	89	66	23	94.0	28.1	0.167	24
4	0	137	40	35	168.0	43.1	2.288	33
...	...	...	...	...	...	...	...	...
763	10	101	76	48	180.0	32.9	0.171	63
764	2	122	70	27	NaN	36.8	0.340	25
765	5	121	72	23	112.0	26.2	0.245	30
766	1	126	60	0	NaN	30.1	0.349	41
767	1	93	70	31	NaN	30.4	0.315	23

768 rows × 9 columns

In [15]:

df['Insulin'] = df['Insulin'].fillna(df['Insulin'].median()) *#fill NAN to median*

In [16]:

df

Out[16]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72	35	125.0	33.6	0.627	50
1	1	85	66	29	125.0	26.6	0.351	31
2	8	183	64	0	125.0	23.3	0.672	31
3	1	89	66	23	94.0	28.1	0.167	21
4	0	137	40	35	168.0	43.1	2.288	31
...	...	...	...	...	...	...	...	...
763	10	101	76	48	180.0	32.9	0.171	61
764	2	122	70	27	125.0	36.8	0.340	21
765	5	121	72	23	112.0	26.2	0.245	30
766	1	126	60	0	125.0	30.1	0.349	41
767	1	93	70	31	125.0	30.4	0.315	21

768 rows × 9 columns

In [17]:

df['Insulin'].isnull().sum() #check if it has any NAN

Out[17]: np.int64(0)

In [18]:

df['Insulin'].nunique()

Out[18]: 185

In [19]:

df['Insulin'].describe()

Out[19]: count 768.000000
mean 140.671875
std 86.383060
min 14.000000
25% 121.500000
50% 125.000000
75% 127.250000
max 846.000000
Name: Insulin, dtype: float64

In [20]:

df.columns

Out[20]: Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
dtype='object')

In [21]:

(df['Glucose'] == 0).sum()

Out[21]: np.int64(5)

In [22]:

(df['Age'] == 0).sum()

Out[22]: np.int64(0)

In [23]:

(df['BloodPressure'] == 0).sum()

Out[23]: np.int64(35)

```
In [24]: df['BloodPressure'] = df['BloodPressure'].replace(0, np.nan)
```

```
In [25]: df['BloodPressure'] = df['BloodPressure'].fillna(df['BloodPressure'].median())
```

```
In [26]: df
```

Out[26]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72.0	35	125.0	33.6	0.627	50
1	1	85	66.0	29	125.0	26.6	0.351	31
2	8	183	64.0	0	125.0	23.3	0.672	33
3	1	89	66.0	23	94.0	28.1	0.167	24
4	0	137	40.0	35	168.0	43.1	2.288	33
...	...	...	...	...	...	...	...	...
763	10	101	76.0	48	180.0	32.9	0.171	63
764	2	122	70.0	27	125.0	36.8	0.340	27
765	5	121	72.0	23	112.0	26.2	0.245	30
766	1	126	60.0	0	125.0	30.1	0.349	47
767	1	93	70.0	31	125.0	30.4	0.315	23

768 rows × 9 columns



```
In [27]: (df['BloodPressure'] == 0).sum()
```

```
Out[27]: np.int64(0)
```

```
In [28]: (df['SkinThickness'] == 0).sum()
```

```
Out[28]: np.int64(227)
```

```
In [29]: df['SkinThickness'] = df['SkinThickness'].replace(0, np.nan)
```

```
In [30]: df['SkinThickness'] = df['SkinThickness'].fillna(df['SkinThickness'].median())
```

```
In [31]: (df['SkinThickness'] == 0).sum()
```

```
Out[31]: np.int64(0)
```

```
In [32]: df
```

Out[32]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72.0	35.0	125.0	33.6	0.627	50
1	1	85	66.0	29.0	125.0	26.6	0.351	31
2	8	183	64.0	29.0	125.0	23.3	0.672	30
3	1	89	66.0	23.0	94.0	28.1	0.167	21
4	0	137	40.0	35.0	168.0	43.1	2.288	33
...	...	...	...	...	...	...	...	...
763	10	101	76.0	48.0	180.0	32.9	0.171	63
764	2	122	70.0	27.0	125.0	36.8	0.340	21
765	5	121	72.0	23.0	112.0	26.2	0.245	30
766	1	126	60.0	29.0	125.0	30.1	0.349	41
767	1	93	70.0	31.0	125.0	30.4	0.315	21

768 rows × 9 columns



In [33]:

```
(df['BMI'] == 0).sum()
```

Out[33]: np.int64(11)

In [34]:

```
df['BMI'] = df['BMI'].replace(0, np.nan)
```

In [35]:

```
df['BMI'] = df['BMI'].fillna(df['BMI'].median())
```

In [36]:

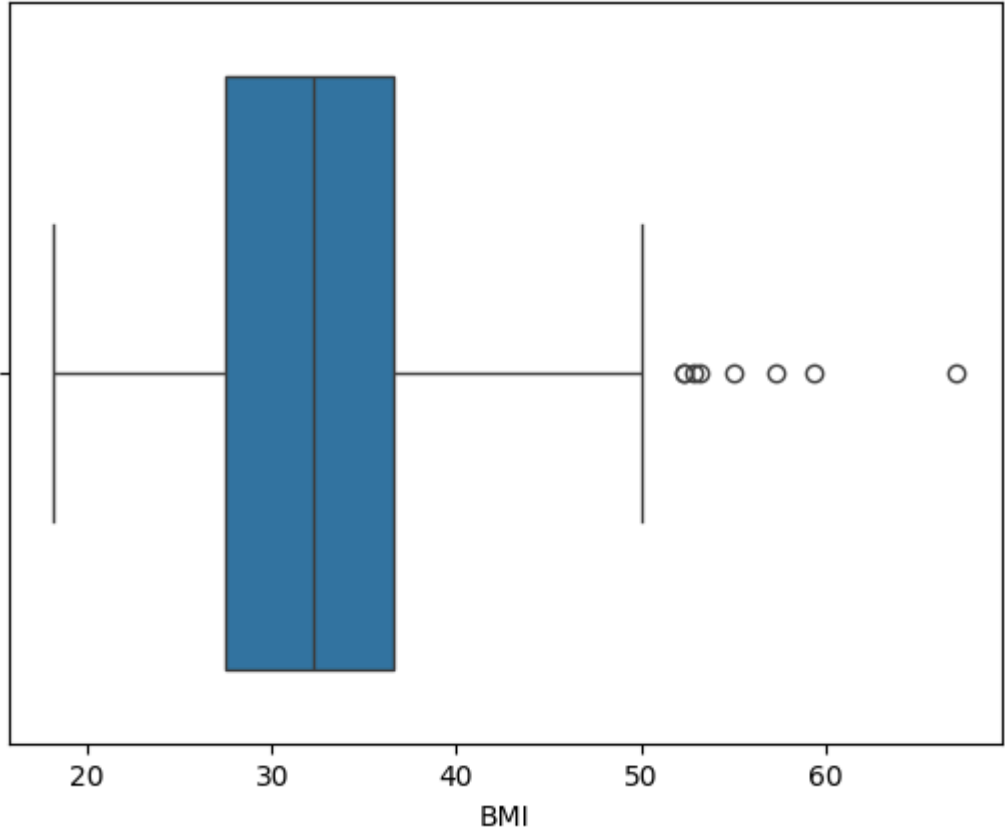
```
(df['BMI'] == 0).sum()
```

Out[36]: np.int64(0)

In [37]:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(x=df['BMI'])
plt.show()
```

```
In [38]: df.describe()
```

Out[38]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigree
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	72.386719	29.108073	140.671875	32.455208	
std	3.369578	31.972618	12.096642	8.791221	86.383060	6.875177	
min	0.000000	0.000000	24.000000	7.000000	14.000000	18.200000	
25%	1.000000	99.000000	64.000000	25.000000	121.500000	27.500000	
50%	3.000000	117.000000	72.000000	29.000000	125.000000	32.300000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	

```
In [39]: df
```

Out[39]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
0	6	148	72.0	35.0	125.0	33.6	0.627	50
1	1	85	66.0	29.0	125.0	26.6	0.351	31
2	8	183	64.0	29.0	125.0	23.3	0.672	30
3	1	89	66.0	23.0	94.0	28.1	0.167	21
4	0	137	40.0	35.0	168.0	43.1	2.288	33
...	...	...	...	...	...	...	...	...
763	10	101	76.0	48.0	180.0	32.9	0.171	63
764	2	122	70.0	27.0	125.0	36.8	0.340	21
765	5	121	72.0	23.0	112.0	26.2	0.245	30
766	1	126	60.0	29.0	125.0	30.1	0.349	41
767	1	93	70.0	31.0	125.0	30.4	0.315	21

768 rows × 9 columns

In [40]:

```
df['Insulin'].nunique()
```

Out[40]: 185

In [41]:

```
df.isnull().sum()
```

Out[41]: Pregnancies 0
Glucose 0
BloodPressure 0
SkinThickness 0
Insulin 0
BMI 0
DiabetesPedigreeFunction 0
Age 0
Outcome 0
dtype: int64

In [42]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 768 entries, 0 to 767  
Data columns (total 9 columns):  
#   Column                Non-Null Count  Dtype  
---  -  
0   Pregnancies           768 non-null    int64  
1   Glucose               768 non-null    int64  
2   BloodPressure         768 non-null    float64  
3   SkinThickness         768 non-null    float64  
4   Insulin               768 non-null    float64  
5   BMI                   768 non-null    float64  
6   DiabetesPedigreeFunction 768 non-null    float64  
7   Age                   768 non-null    int64  
8   Outcome               768 non-null    int64  
dtypes: float64(5), int64(4)  
memory usage: 54.1 KB
```

In [43]:

```
df['BloodPressure'] = df['BloodPressure'].astype('int')
```

In [44]:

```
df['SkinThickness'] = df['SkinThickness'].astype('int')
```

```
In [45]: df['Insulin'] = df['Insulin'].astype('int')
```

```
In [46]: # df['BMI'] = df['BMI'].astype('int')
```

```
In [47]: # df['DiabetesPedigreeFunction'] = df['DiabetesPedigreeFunction'].astype('int')
```

```
In [48]: # df.info()
```

Separate features I/P= (X) and target (y)

```
In [49]: X = df.drop('Outcome', axis=1) #feature data/ input data  
Y = df['Outcome'] #output data
```

```
In [50]: print(X.shape)  
print(Y.shape)
```

```
(768, 8)
```

```
(768,)
```

Split into Train and Test

```
In [51]: from sklearn.model_selection import train_test_split  
  
X_train, X_test, Y_train, Y_test = train_test_split(  
    X, Y, test_size = 0.2, random_state = 42  
)
```

```
In [52]: print(X_train.shape)  
print(X_test.shape)
```

```
(614, 8)
```

```
(154, 8)
```

Scaling Converting big number to small (for fair learning)

```
In [53]: from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train) #fit. is learns statistics  
X_test = scaler.transform(X_test) #transform is applies those learned values to scale data.
```

```
In [54]: from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression(max_iter=200)  
model.fit(X_train, Y_train)
```

```
Out[54]: ▼ LogisticRegression ⓘ ?
```

```
► Parameters
```

```
In [55]: y_pred = model.predict(X_test)  
#now predict the I/p with the help of LogisticRegression
```

```
In [56]: from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(Y_test, y_pred)  
print("Model Accuracy:", accuracy)
```

Model Accuracy: 0.7532467532467533

```
In [57]: '''After feature scaling and Logistic Regression, the model achieved an accuracy
of approximately 74%,
which indicates prediction performance on unseen data.
'''
```

```
Out[57]: 'After feature scaling and Logistic Regression, the model achieved an accuracy \nof approxima
tely 74%,\nwhich indicates prediction performance on unseen data.\n'
```

```
In [58]: print("\nEnter Patient Details:")

preg = int(input("Pregnancies: "))

glucose = float(input("Glucose Level:"))

bp = float(input("Blood Pressure: "))

skin = float(input("Skin Thickness:"))

insulin = float(input("Insulin:"))

bmi = float(input("BMI: "))

dpf = float(input("Diabetes Pedigree Function: "))

age = int(input("Age: "))

import pandas as pd

columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
           'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

# Step 1: create simple List
input_data = [preg, glucose, bp, skin, insulin, bmi, dpf, age]

# Step 2: convert to DataFrame (ONLY once)
input_df = pd.DataFrame([input_data], columns=columns)

# Step 3: scale
input_scaled = scaler.transform(input_df)

# Step 4: predict
result = model.predict(input_scaled)

if result == 1:
    print("yes")
else:
    print("NO")
```

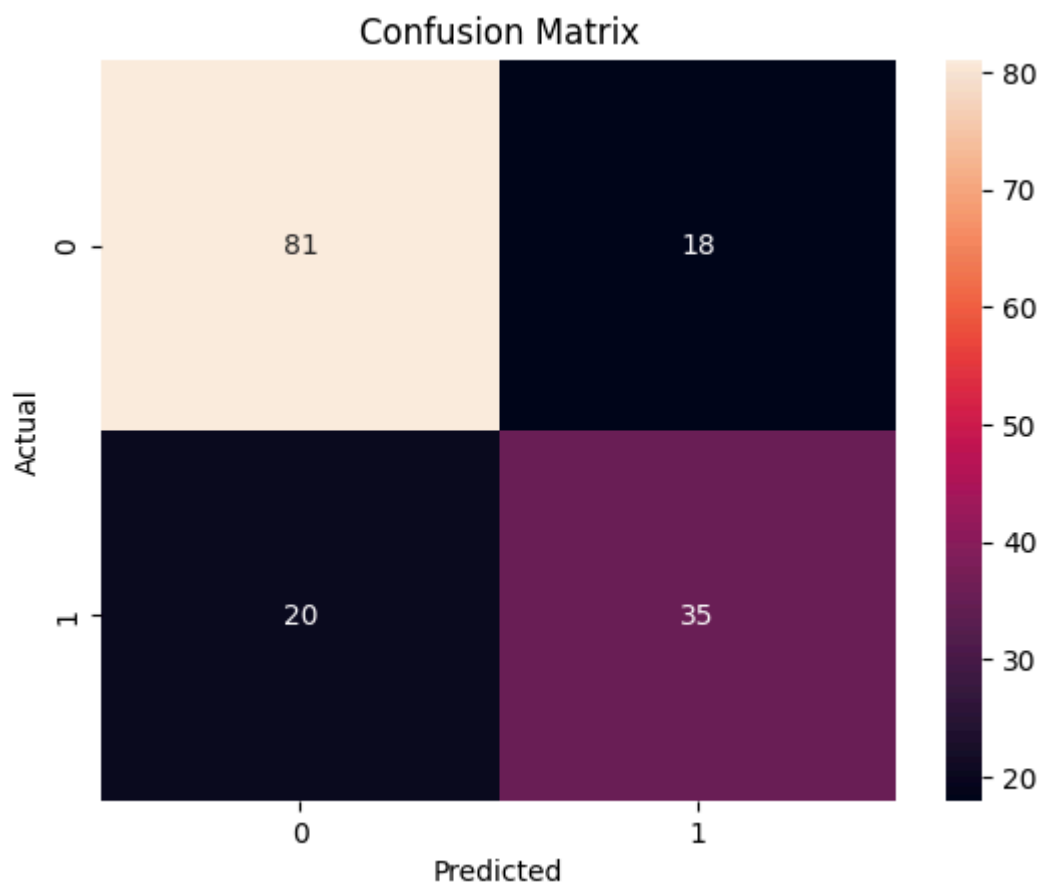
Enter Patient Details:

yes

```
In [59]: from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(Y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```



```
In [60]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load Data
df = pd.read_csv('D:/ML Project/diabetes.csv')

# 2. Basic Cleaning: Replace '0' values in medical columns with the mean
# (A blood pressure of 0 is impossible, so we treat it as missing data)
cols_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in cols_with_zeros:
    df[col] = df[col].replace(0, df[col].mean())

# 3. Define Features (X) and Target (y)
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# 4. Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Scale Data (Important for SVM and KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Initialize Models
models = {
    "Logistic Regression": LogisticRegression(),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Support Vector Machine": SVC(),
    "K-Nearest Neighbors": KNeighborsClassifier(),
```

```

    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

# 7. Train and Evaluate
results = []
print("Training models...")
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    results.append({"Model": name, "Accuracy": acc})

# 8. Create Comparison Table
results_df = pd.DataFrame(results).sort_values(by="Accuracy", ascending=False)
print("\nModel Comparison Table:")
print(results_df)

# 9. Plot Results
plt.figure(figsize=(10, 6))
sns.barplot(x="Accuracy", y="Model", data=results_df, palette="viridis")
plt.title("Model Comparison: Accuracy Score")
plt.xlim(0, 1.0) # Set x-axis limit from 0 to 1
plt.show()

```

Training models...

Model Comparison Table:

	Model	Accuracy
0	Logistic Regression	0.766234
1	Random Forest	0.766234
3	K-Nearest Neighbors	0.766234
4	Gradient Boosting	0.766234
2	Support Vector Machine	0.733766

C:\Users\Adwait\AppData\Local\Temp\ipykernel_22496\2499066634.py:58: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Accuracy", y="Model", data=results_df, palette="viridis")
```

